

VLIW Bundle Density: Where the Limit Is, and What Optimizations Remain

APARA C Compiler — Optimization Status and Analysis

APARA C Compiler Project

June 26, 2026

1 Summary

After five optimization passes (register caching, storage-class alias analysis, common subexpression elimination, instruction scheduling, and loop-invariant code motion), the compiler reaches an average of about **2.1 real instructions per bundle**, with the parallel regions (vector ALU and dot-product kernels) already filling the **full 8-wide bundle**. This document explains, for the examination, *why* the average is what it is, whether it can be pushed substantially higher, and which optimization directions remain worthwhile. The short conclusion: **bundle density is now close to the architectural ceiling, and the productive direction for further speed-up is reducing the number of executed loads/stores, not packing the bundles tighter.**

2 What Determines Instructions-per-Bundle

The number of instructions packed into a VLIW bundle is governed by three things, only one of which the compiler controls:

1. **Available instruction-level parallelism (ILP)** — how many mutually independent instructions exist to pack. This is a property of the program’s data dependencies, not of the bundler.
2. **The bundler’s ability to exploit that ILP** — our bundler is already a maximal greedy packer *plus* a within-basic-block list scheduler that reorders independent instructions to cluster them. This part is effectively done.
3. **The register file size** — to issue K independent operations in one cycle, their operands and results must all live in distinct registers simultaneously.

The remaining headroom is limited by items (1) and (3), which are fundamental, not by (2).

3 The Two Hard Limits

Limit 1 — the 28-register hardware ceiling

APARA exposes 32 physical registers, of which 28 are allocatable (the rest are the zero register, frame pointer, stack pointer, and global base). To pack K *independent* arithmetic operations into one bundle requires roughly

$$\underbrace{2K}_{\text{operands}} + \underbrace{K}_{\text{results}} = 3K \quad \text{registers live at once.}$$

With 28 registers, this caps the realistically packable independent operations at about **8–9** — which is exactly why the dense regions of our generated code already reach the 8-instruction bundle width. *One cannot pack more independent instructions than there are registers to hold their operands.* This is a property of the hardware and cannot be removed by the compiler.

Limit 2 — genuine serial data dependencies

The matrix-multiply kernel is an intrinsically serial chain:

address $\rightarrow$ wide-load $\rightarrow$ $\$dot$ $\rightarrow$ $\$dot$ $\$accumulate$ $\rightarrow$ store,

where each step needs the previous step’s result. No reordering or packing can parallelize a *single* instance of such a chain; the only parallelism comes from *different, independent* instances (e.g. separate loop iterations), and exploiting those requires more registers (Limit 1) plus register renaming.

Consequently the *average* density (about 2.1) is held down by these serial chains and by inherently-serial setup and control-flow code — **not** by the bundler, which is already maximal. The bundler is performing close to optimally given the register file and the dependency structure.

4 Remaining Density Techniques — and an Honest Verdict

Technique	Can it raise density?	Verdict
Software pipelining	Yes — overlap iteration $i+1$ ’s loads with iteration i ’s compute (how the ISA’s unrolled-dot example reaches 8-wide)	The legitimate way, but needs register renaming and is bounded by the 28-register wall; large, high-risk
Register renaming / SSA-based allocation	Enables unrolling and pipelining without false dependencies	Major allocator rewrite; <i>still</i> capped at 28 registers, so limited headroom
Wider loads ($\$u256$)	Fewer load <i>instructions</i> (more bytes per load)	Concrete but data-layout-dependent; modest gain
Instruction fusion / better selection	Fewer instructions overall	Marginal

Every one of these runs into the same 28-register wall. That is the “dead end” one senses when pushing density — and it is real, but it is the *hardware’s* limit, not the compiler’s. The achievable gains are small relative to the implementation effort and risk (both loop unrolling and the aggressive form of LICM already hit this register-pressure ceiling during development).

5 The Important Reframe: Density vs. Executed Work

“More instructions per bundle” (density) and “faster execution” (fewer executed instructions) are different goals, and they diverged at the LICM stage:

- **Density** is near its architectural ceiling (limited by 28 registers and serial dependencies).
- **Executed work** is *not* near a ceiling. Loop-invariant code motion cut the matrix multiply’s executed loads by **33.5%** ($6441 \rightarrow 4281$) and total executed instructions by **22.7%** ($55201 \rightarrow 42665$) *without changing bundle density at all* — it simply does less work.

The lesson: for further speed-up, the productive direction is **doing less work**, not packing bundles tighter. Reducing executed loads/stores improves runtime even when bundle density stays the same.

6 Recommended Next Steps (in priority order)

1. **Loop-carried register allocation for loop counters** — keep induction variables (i, j, k) in registers across iterations instead of reloading them from the stack every iteration. Pure executed-load reduction; same direction as LICM; does not fight the register wall. *Highest value, lowest risk.*
2. **Strength reduction** — replace multiply/divide/modulo by a power of two with shift / shift / mask. Fewer ALU instructions; safe where signedness permits.
3. **Wider loads (\$u256)** where the data layout allows — fewer load instructions for the same bytes.
4. **(Deferred / high-effort)** Software pipelining with register renaming — the only path to materially higher *density*, but bounded by the 28-register limit and a substantial, higher-risk rewrite. Recommended only if hot-loop throughput is the explicit objective.

7 Conclusion

Bundle density is **not** blocked by a weak bundler; it is bounded by the APARA register file (28 usable registers) and by genuine serial data dependencies. The dense regions already reach the full 8-wide bundle, and the whole-program average of ~ 2.1 is close to optimal under those constraints. Pushing density materially higher would require software pipelining plus register renaming — a large effort with limited headroom because of the 28-register ceiling. The better-value direction, already demonstrated by LICM’s 22–33% reduction in executed work, is to **continue eliminating redundant loads, stores, and recomputation**, which improves runtime independently of bundle density.